

EARTHQUAKE RISK ANALYSIS

PROGRAM 2

R CODE:

give the code ;like this # Install packages once

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

```
set.seed(42)
```

```
n <- 500
```

```
earthquake <- data.frame(
```

```
  Magnitude = runif(n,0.5,9.5),
```

```
  Depth = runif(n,1,700),
```

```
  Latitude = runif(n,-90,90),
```

```
  Longitude = runif(n,-180,180),
```

```
  Aftershock_Frequency = sample(1:1000,n,replace=TRUE)
```

```
)
```

```
ggplot(
```

```
  earthquake,
```

```
  aes(x=Magnitude)
```

```
)+
```

```
  geom_histogram(
```

```
    bins=30,
```

```
    fill="blue",
```

```
    color="black"
```

```
)+
```

```
  labs(
```

```
title="Earthquake Magnitude Distribution (Linear Scale)",
x="Magnitude",
y="Frequency"
)+
theme_minimal()
ggplot(
earthquake,
aes(x=Magnitude)
)+
geom_histogram(
bins=30,
fill="red",
color="black"
)+
scale_y_log10()+
labs(
title="Earthquake Magnitude Distribution (Log Scale)",
x="Magnitude",
y="Frequency (Log Scale)"
)+
theme_minimal()
ggplot(
earthquake,
aes(
x=Depth,
```

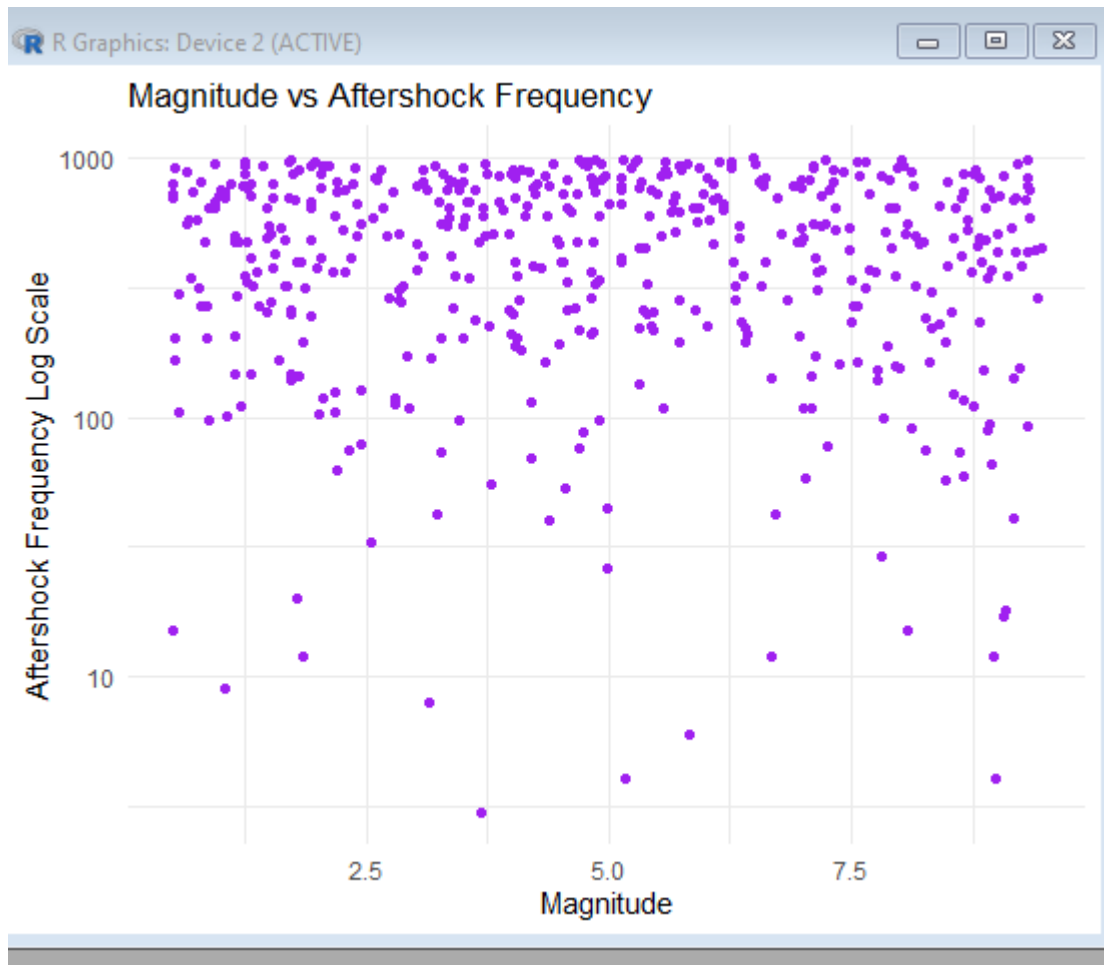
```
y=Magnitude,  
color=Magnitude  
)  
)+  
geom_point(size=3)+  
scale_color_gradient(  
low="yellow",  
high="red"  
)+  
labs(  
title="Depth vs Magnitude",  
x="Depth (km)",  
y="Magnitude"  
)+  
theme_minimal()  
ggplot(  
earthquake,  
aes(  
x=Depth,  
y=Magnitude,  
color=Magnitude  
)  
)+  
geom_point(size=3)+  
scale_x_log10()+
```

```
scale_color_gradient(
  low="blue",
  high="red"
)+
labs(
  title="Depth vs Magnitude (Log Scale)",
  x="Depth (km) Log Scale",
  y="Magnitude"
)+
theme_minimal()
ggplot(
  earthquake,
  aes(
    x=Magnitude,
    y=Aftershock_Frequency
  )
)+
geom_point(
  color="purple"
)+
scale_y_log10()+
labs(
  title="Magnitude vs Aftershock Frequency",
  x="Magnitude",
  y="Aftershock Frequency Log Scale"
```

)+

theme_minimal()

OUTPUT:



WIND DIRECTION ANALYSIS FOR AIRPORT OPERATIONS PROGRAM 3

R PROGRAM:

```
install.packages("ggplot2")

library(ggplot2)

set.seed(42)

n <- 8760

wind <- data.frame(
  WindSpeed = runif(n, 0, 40),
  WindDirection = runif(n, 0, 360)
)

wind$Direction <- cut(
  wind$WindDirection,
  breaks = seq(0, 360, by = 22.5),
  include.lowest = TRUE,
  labels = c(
    "N", "NNE", "NE", "ENE",
    "E", "ESE", "SE", "SSE",
    "S", "SSW", "SW", "WSW",
    "W", "WNW", "NW", "NNW"
  )
)

wind_frequency <- as.data.frame(table(wind$Direction))
colnames(wind_frequency) <- c("Direction", "Frequency")

ggplot(
  wind_frequency,
```

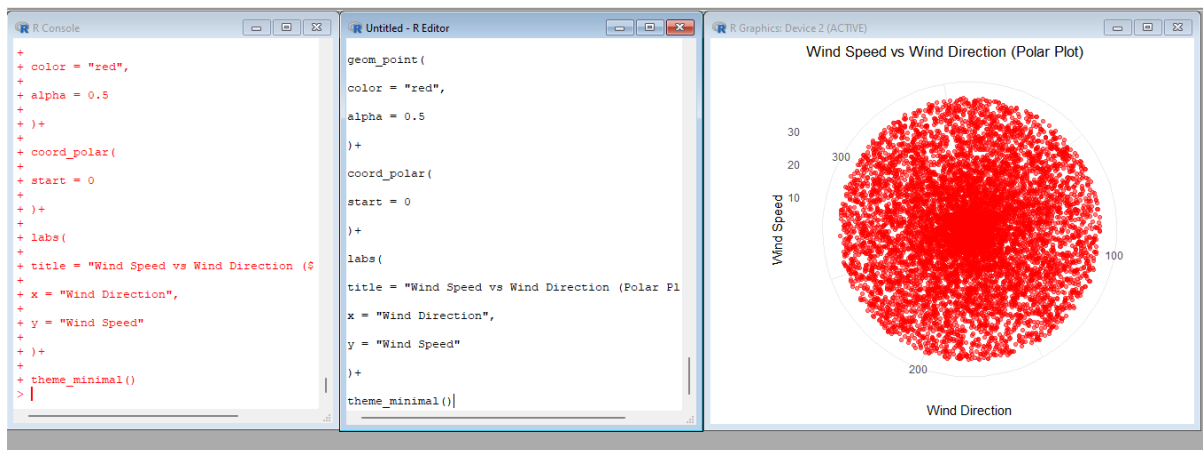
```
aes(  
  x = Direction,  
  y = Frequency  
)  
) +  
  geom_bar(  
    stat = "identity",  
    fill = "steelblue",  
    color = "black"  
) +  
  labs(  
    title = "Wind Direction Frequency (Cartesian Plot)",  
    x = "Wind Direction",  
    y = "Frequency"  
) +  
  theme_minimal()  
ggplot(  
  wind_frequency,  
  aes(  
    x = Direction,  
    y = Frequency,  
    fill = Frequency  
  )  
) +  
  geom_bar(  
    stat = "identity",  
    fill = "steelblue",  
    color = "black"  
) +  
  labs(  
    title = "Wind Direction Frequency (Cartesian Plot)",  
    x = "Wind Direction",  
    y = "Frequency"  
) +  
  theme_minimal()
```

```
stat = "identity",  
color = "black"  
) +  
coord_polar(  
start = 0  
) +  
labs(  
title = "Wind Rose (Polar Coordinate Plot)",  
x = "Wind Direction",  
y = "Frequency"  
) +  
theme_minimal()  
ggplot(  
wind,  
aes(  
x = WindDirection,  
y = WindSpeed  
)  
) +  
geom_point(  
color = "darkgreen",  
alpha = 0.6  
) +  
labs(  
title = "Wind Speed vs Wind Direction",
```



```
x = "Wind Direction (Degrees)",  
y = "Wind Speed (km/h)"  
)+  
theme_minimal()  
ggplot(  
  wind,  
  aes(  
    x = WindDirection,  
    y = WindSpeed  
  )  
  
)+  
  geom_point(  
    color = "red",  
    alpha = 0.5  
  )+  
  coord_polar(  
    start = 0  
  )+  
  labs(  
    title = "Wind Speed vs Wind Direction (Polar Plot)",  
    x = "Wind Direction",  
    y = "Wind Speed"  
  )+  
  theme_minimal()
```

OUTPUT:



GLOBAL CLIMATE CHANGE STUDY

PROGRAM 5

R PROGRAM:

```
install.packages("ggplot2")  
library(ggplot2)  
set.seed(42)  
countries <- paste("Country", 1:180)  
years <- 1980:2025  
climate <- expand.grid(  
  Country = countries,  
  Year = years  
)  
climate$Temperature_Anomaly <- round(  
  rnorm(  
    nrow(climate),  
    mean = 0,  
    sd = 1  
  ),  
  2  
)  
ggplot(  
  climate,  
  aes(  
    x = Year,  
    y = Country,
```

```

    fill = Temperature_Anomaly
  )
) +
geom_tile() +
scale_fill_gradient2(
  low = "blue",
  mid = "white",
  high = "red",
  midpoint = 0
) +
labs(
  title = "Temperature Anomalies (Blue-White-Red)",
  x = "Year",
  y = "Country",
  fill = "Anomaly (°C)"
) +
theme_minimal()
ggplot(
  climate,
  aes(
    x = Year,
    y = Country,
    fill = Temperature_Anomaly
  )
) +

```

```

geom_tile() +
scale_fill_gradient2(
  low = "darkgreen",
  mid = "white",
  high = "purple",
  midpoint = 0
) +
labs(
  title = "Temperature Anomalies (Green-White-Purple)",
  x = "Year",
  y = "Country",
  fill = "Anomaly (°C)"
) +
theme_minimal()
ggplot(
  climate,
  aes(
    x = Year,
    y = Country,
    fill = Temperature_Anomaly
  )
) +
geom_tile() +
scale_fill_gradient2(
  low = "navy",

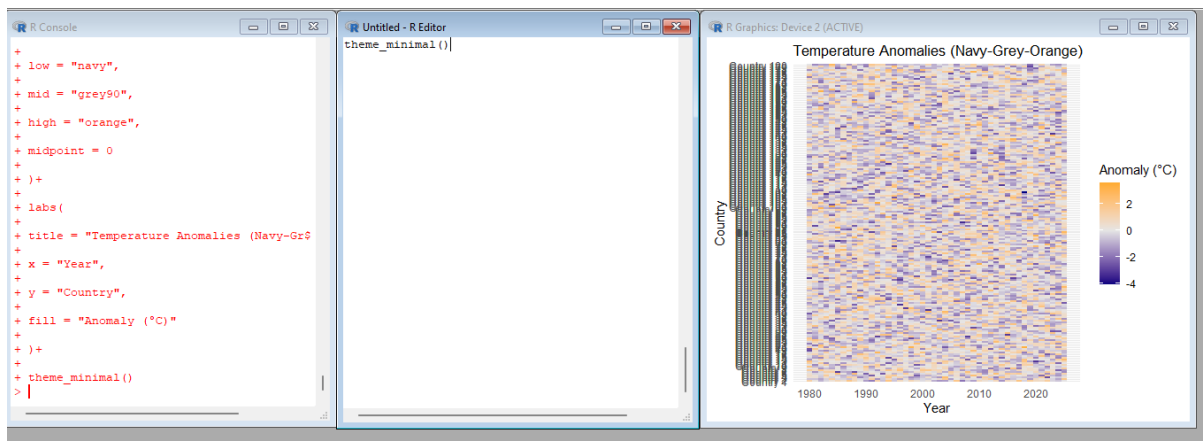
```

```

mid = "grey90",
high = "orange",
midpoint = 0
)+
labs(
  title = "Temperature Anomalies (Navy-Grey-Orange)",
  x = "Year",
  y = "Country",
  fill = "Anomaly (°C)"
)+
theme_minimal()

```

OUTPUT



PROGRAM 7

R CODE

Install package (Run once)

```
# install.packages("ggplot2")
```

```
library(ggplot2)
```

```
# -----
```

```
# Generate NDVI Dataset
```

```
# -----
```

```
set.seed(42)
```

```
rows <- 50
```

```
cols <- 50
```

```
ndvi <- matrix(  
  runif(rows * cols, min = -1, max = 1),  
  nrow = rows,  
  ncol = cols  
)
```

```
# Convert matrix to dataframe
```

```
df <- expand.grid(  
  X = 1:rows,  
  Y = 1:cols  
)
```

```
df$NDVI <- as.vector(ndvi)
```

```
# -----  
# Heatmap 1 - Greens  
# -----  
ggplot(df, aes(X, Y, fill = NDVI)) +  
  geom_tile() +  
  scale_fill_gradient(  
    low = "white",  
    high = "darkgreen"  
  ) +  
  labs(title = "NDVI Heatmap - Greens") +  
  theme_minimal()
```

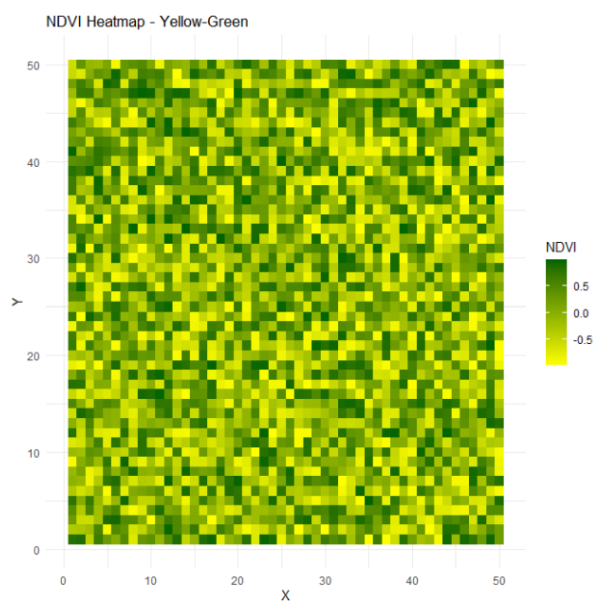
```
# -----  
# Heatmap 2 - Yellow-Green  
# -----  
ggplot(df, aes(X, Y, fill = NDVI)) +  
  geom_tile() +  
  scale_fill_gradient(  
    low = "yellow",  
    high = "darkgreen"  
  ) +  
  labs(title = "NDVI Heatmap - Yellow-Green") +  
  theme_minimal()
```

```
# -----  
# Heatmap 3 - Red-Yellow-Green
```



```
# -----
ggplot(df, aes(X, Y, fill = NDVI)) +
  geom_tile() +
  scale_fill_gradient2(
    low = "red",
    mid = "yellow",
    high = "green",
    midpoint = 0
  ) +
  labs(title = "NDVI Heatmap - Red Yellow Green") +
  theme_minimal()
```

OUTPUT



PROGRAM 8

R CODE

```
# -----  
# Generate Cryptocurrency Dataset  
# -----  
set.seed(42)  
  
days <- 1:1000  
  
bitcoin <- 1000 * exp(cumsum(rnorm(1000, 0.001, 0.02)))  
ethereum <- 100 * exp(cumsum(rnorm(1000, 0.0012, 0.025)))  
solana <- 10 * exp(cumsum(rnorm(1000, 0.0015, 0.03)))  
cardano <- 1 * exp(cumsum(rnorm(1000, 0.0018, 0.035)))  
  
crypto <- data.frame(  
  Day = days,  
  Bitcoin = bitcoin,  
  Ethereum = ethereum,  
  Solana = solana,  
  Cardano = cardano  
)  
  
# -----  
# Cartesian Plot  
# -----  
matplot(  
  crypto$Day,  
  crypto[,2:5],
```

```
type = "l",  
lty = 1,  
lwd = 2,  
col = c("orange", "blue", "green", "red"),  
xlab = "Days",  
ylab = "Price (USD)",  
main = "Cryptocurrency Prices (Cartesian Scale)"  
)
```

```
legend(  
  "topleft",  
  legend = c("Bitcoin", "Ethereum", "Solana", "Cardano"),  
  col = c("orange", "blue", "green", "red"),  
  lty = 1,  
  lwd = 2  
)
```

```
# -----  
# Logarithmic Scale Plot  
# -----
```

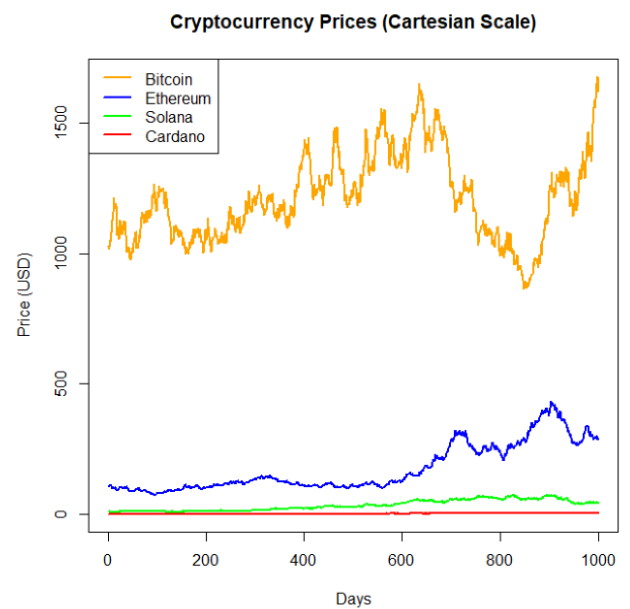
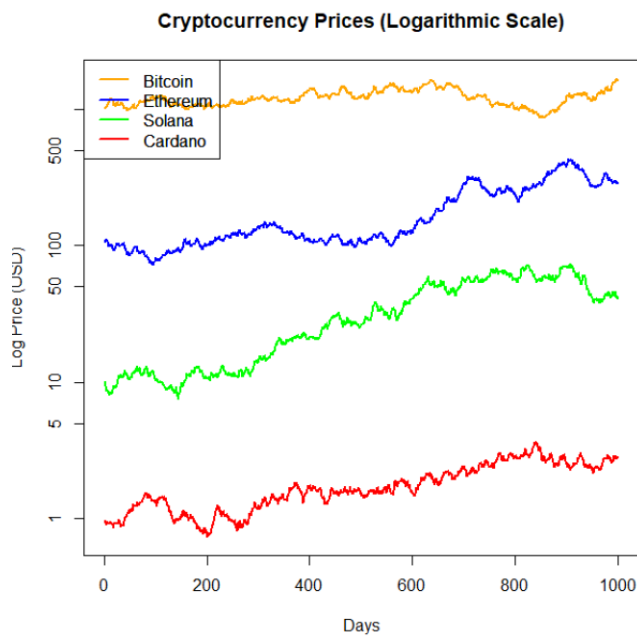
```
matplot(  
  crypto$Day,  
  crypto[,2:5],  
  type = "l",  
  log = "y",  
  lty = 1,  
  lwd = 2,
```

```

col = c("orange", "blue", "green", "red"),
xlab = "Days",
ylab = "Log Price (USD)",
main = "Cryptocurrency Prices (Logarithmic Scale)"
)
legend(
  "topleft",
  legend = c("Bitcoin", "Ethereum", "Solana", "Cardano"),
  col = c("orange", "blue", "green", "red"),
  lty = 1,
  lwd = 2
)

```

OUTPUT



PROGRAM 9

R CODE

```
# -----  
# Generate Marathon Dataset  
# -----  
set.seed(42)  
  
n <- 500  
  
Distance <- seq(0, 42.195, length.out = n)  
Speed <- runif(n, 8, 18)  
HeartRate <- sample(110:190, n, replace = TRUE)  
Elevation <- sample(0:300, n, replace = TRUE)  
Direction <- runif(n, 0, 360)  
  
marathon <- data.frame(  
  Distance,  
  Speed,  
  HeartRate,  
  Elevation,  
  Direction  
)  
  
# -----  
# Cartesian Plot  
# -----  
library(ggplot2)
```

```

ggplot(marathon,
  aes(x = Distance,
      y = HeartRate,
      color = Speed)) +
geom_point(size = 2) +
scale_color_gradient(low = "blue", high = "red") +
labs(
  title = "Heart Rate vs Distance",
  x = "Distance (km)",
  y = "Heart Rate (bpm)"
) +
theme_minimal()

```

```

# -----
# Polar Coordinate Plot
# -----

```

```

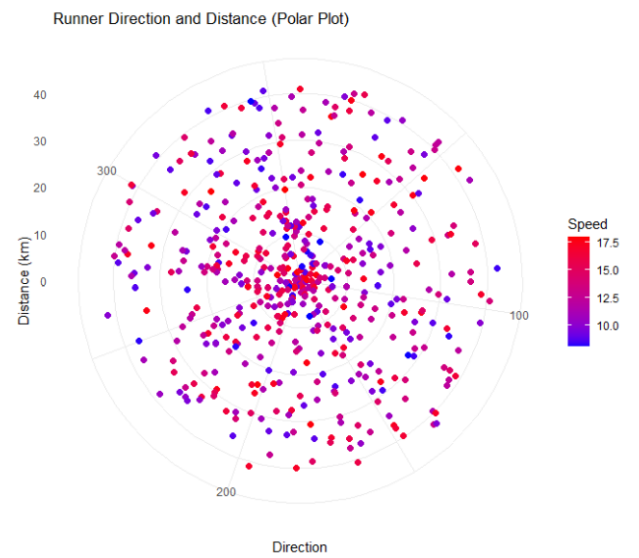
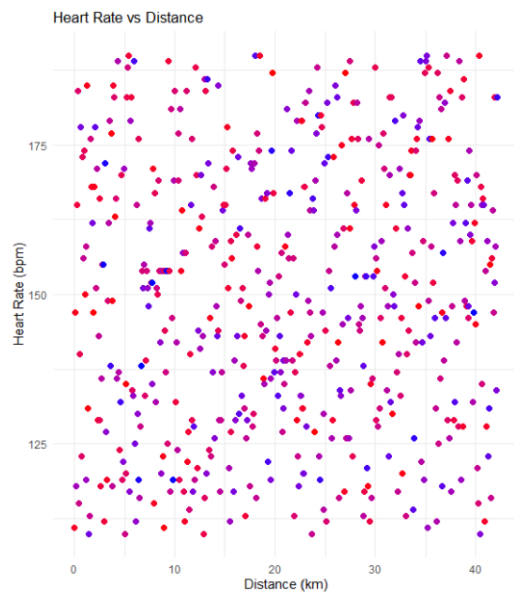
ggplot(marathon,
  aes(x = Direction,
      y = Distance,
      color = Speed)) +
geom_point(size = 2) +
coord_polar() +
scale_color_gradient(low = "blue", high = "red") +
labs(
  title = "Runner Direction and Distance (Polar Plot)",
  x = "Direction",
  y = "Distance (km)"
)

```

) +

theme_minimal()

OUTPUT



PROGRAM 2

PYTHON CODE AND OUTPUT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
n = 500
df = pd.DataFrame({
    "Magnitude": np.round(np.random.uniform(0.5, 9.5, n), 1),
    "Depth": np.random.randint(1, 701, n),
    "Latitude": np.round(np.random.uniform(-90, 90, n), 4),
    "Longitude": np.round(np.random.uniform(-180, 180, n), 4)
})
df["Aftershock_Frequency"] = (
    (df["Magnitude"]**3) +
    np.random.randint(1, 20, n)
).astype(int)
print(df.head())
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Magnitude",
    y="Aftershock_Frequency",
    hue="Depth",
    palette="viridis"
)
```



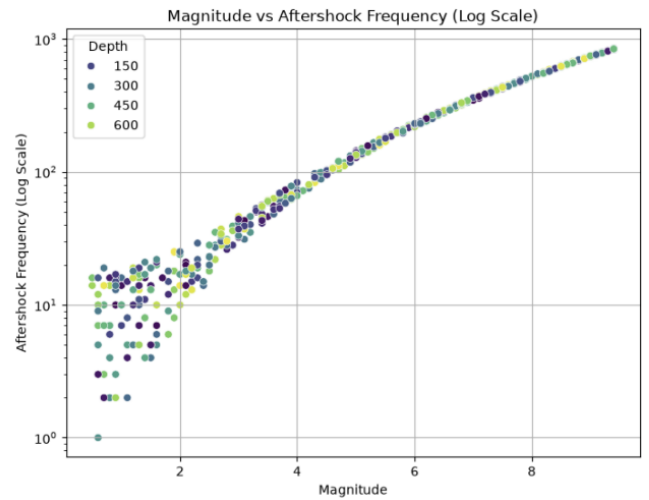
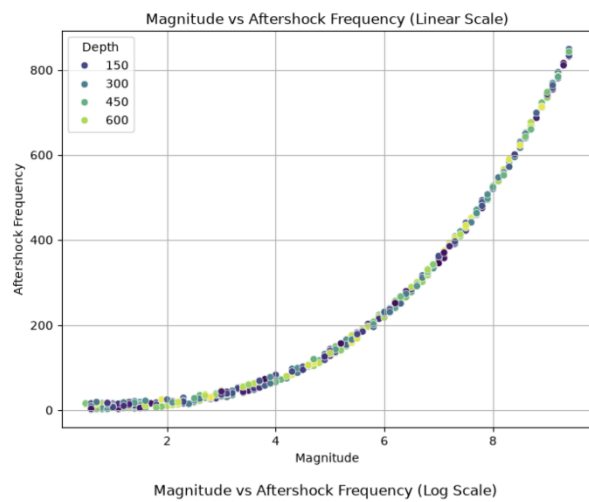
```
plt.title("Magnitude vs Aftershock Frequency (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="Magnitude",
    y="Aftershock_Frequency",
    hue="Depth",
    palette="viridis"
)
plt.yscale("log")
plt.title("Magnitude vs Aftershock Frequency (Log Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency (Log Scale)")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,5))
plt.hist(df["Magnitude"], bins=15, color="skyblue", edgecolor="black")
plt.title("Earthquake Magnitude Distribution")
plt.xlabel("Magnitude")
plt.ylabel("Count")
plt.show()
plt.figure(figsize=(8,5))
plt.hist(
```

```

df["Magnitude"],
bins=15,
color="orange",
edgecolor="black",
log=True
)
plt.title("Magnitude Distribution (Log Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Log Count")
plt.show()

```

OUTPUT



PROGRAM 3

PYTHON CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
n = 365 # One year (daily observations)
wind_speed = np.random.uniform(5, 35, n) # km/h
wind_direction = np.random.uniform(0, 360, n) # degrees

df = pd.DataFrame({
    "Wind_Speed": wind_speed,
    "Wind_Direction": wind_direction
})
print(df.head())
plt.figure(figsize=(8,5))
plt.scatter(df["Wind_Direction"],
            df["Wind_Speed"],
            color="blue")

plt.title("Wind Speed vs Wind Direction (Cartesian Plot)")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed (km/h)")
plt.grid(True)
plt.show()
theta = np.deg2rad(df["Wind_Direction"])
plt.figure(figsize=(8,8))
ax = plt.subplot(111, polar=True)
```

```

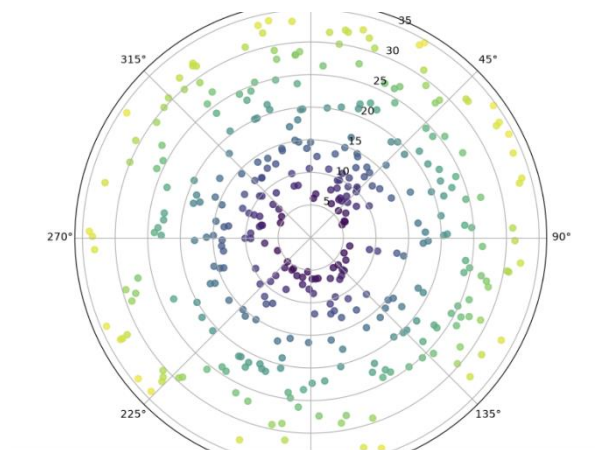
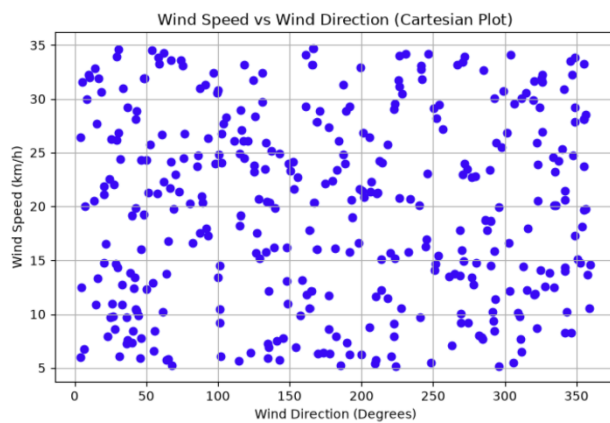
ax.scatter(theta,
           df["Wind_Speed"],
           c=df["Wind_Speed"],
           cmap="viridis",
           alpha=0.75)

ax.set_theta_zero_location("N") # North at top
ax.set_theta_direction(-1)     # Clockwise

ax.set_title("Wind Rose (Polar Coordinate Plot)")
plt.show()

```

OUTPUT



PROGRAM 5

PYTHON CODE AND OUTPUT

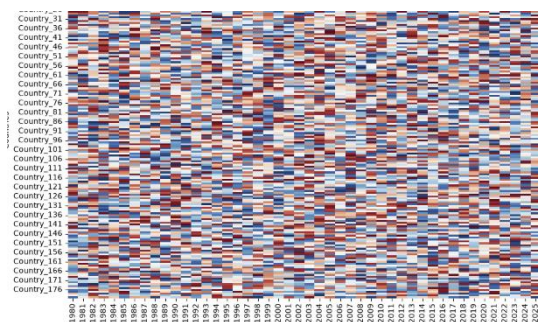
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
countries = [f"Country_{i}" for i in range(1, 181)]
years = list(range(1980, 2026))
data = np.random.uniform(-3, 3, (len(countries), len(years)))
df = pd.DataFrame(data, index=countries, columns=years)
plt.figure(figsize=(14,8))
sns.heatmap(df,
            cmap="coolwarm",
            center=0,
            cbar_kws={"label":"Temperature Anomaly (°C)"}))

plt.title("Global Temperature Anomalies (Coolwarm)")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.show()
plt.figure(figsize=(14,8))
sns.heatmap(df,
            cmap="RdBu_r",
            center=0,
            cbar_kws={"label":"Temperature Anomaly (°C)"}))
```

```
plt.title("Global Temperature Anomalies (RdBu)")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.show()
plt.figure(figsize=(14,8))
sns.heatmap(df,
            cmap="PiYG",
            center=0,
            cbar_kws={"label":"Temperature Anomaly (°C)"}))
```

```
plt.title("Global Temperature Anomalies (PiYG)")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.show()
```

OUTPUT



PROGRAM 7

PYTHON CODE

```
# Import libraries

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

# NDVI values range from -1 to +1
ndvi = np.random.uniform(-1, 1, (50, 50))
plt.figure(figsize=(8,6))
sns.heatmap(ndvi,
            cmap="Greens",
            cbar_kws={"label":"NDVI"})

plt.title("NDVI Heatmap - Greens Palette")
plt.show()

# -----
# Heatmap 2 - Sequential (YlGn)
# -----

plt.figure(figsize=(8,6))
sns.heatmap(ndvi,
            cmap="YlGn",
            cbar_kws={"label":"NDVI"})

plt.title("NDVI Heatmap - Yellow-Green Palette")
plt.show()
```

```

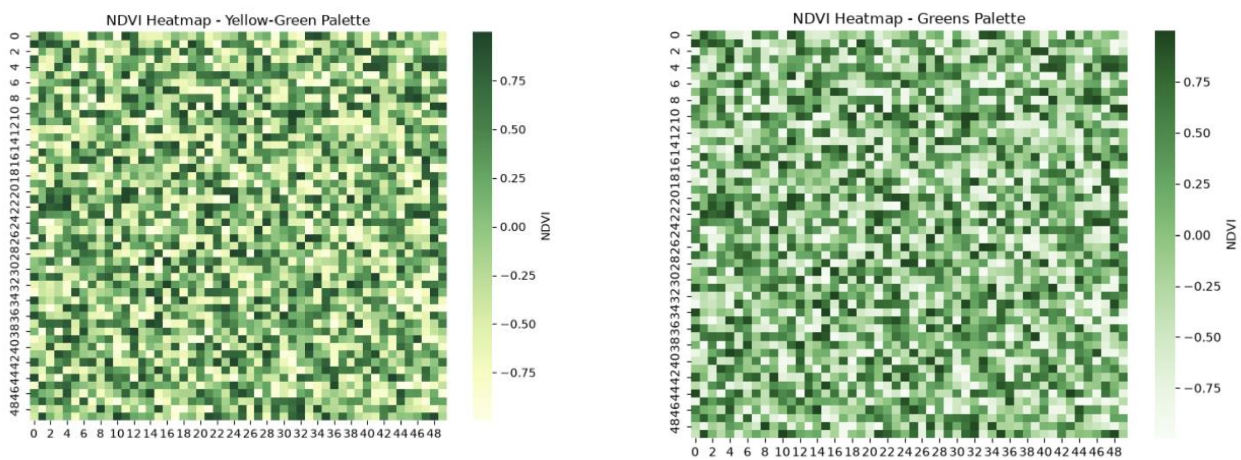
# -----
# Heatmap 3 - Diverging (RdYlGn)
# -----

plt.figure(figsize=(8,6))
sns.heatmap(ndvi,
            cmap="RdYlGn",
            center=0,
            cbar_kws={"label":"NDVI"})

plt.title("NDVI Heatmap - Red Yellow Green")
plt.show()

```

OUTPUT



PROGRAM 8

PYTHON CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
days = np.arange(1, 1001)
bitcoin = 1000 * np.exp(np.cumsum(np.random.normal(0.001, 0.02,
1000)))
ethereum = 100 * np.exp(np.cumsum(np.random.normal(0.0012, 0.025,
1000)))
solana = 10 * np.exp(np.cumsum(np.random.normal(0.0015, 0.03, 1000)))
cardano = 1 * np.exp(np.cumsum(np.random.normal(0.0018, 0.035,
1000)))
df = pd.DataFrame({
    "Day": days,
    "Bitcoin": bitcoin,
    "Ethereum": ethereum,
    "Solana": solana,
    "Cardano": cardano
})

print(df.head())

plt.figure(figsize=(10,6))

plt.plot(df["Day"], df["Bitcoin"], label="Bitcoin", color="orange")
plt.plot(df["Day"], df["Ethereum"], label="Ethereum", color="blue")
plt.plot(df["Day"], df["Solana"], label="Solana", color="green")
```

```
plt.plot(df["Day"], df["Cardano"], label="Cardano", color="red")
```

```
plt.title("Cryptocurrency Prices (Cartesian Scale)")
```

```
plt.xlabel("Days")
```

```
plt.ylabel("Price (USD)")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(10,6))
```

```
plt.plot(df["Day"], df["Bitcoin"], label="Bitcoin", color="orange")
```

```
plt.plot(df["Day"], df["Ethereum"], label="Ethereum", color="blue")
```

```
plt.plot(df["Day"], df["Solana"], label="Solana", color="green")
```

```
plt.plot(df["Day"], df["Cardano"], label="Cardano", color="red")
```

```
plt.yscale("log")
```

```
plt.title("Cryptocurrency Prices (Logarithmic Scale)")
```

```
plt.xlabel("Days")
```

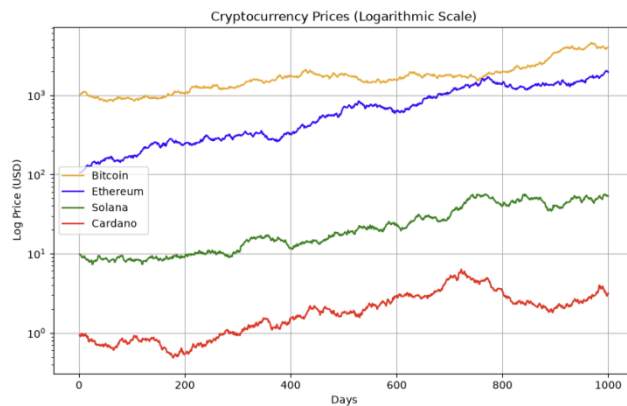
```
plt.ylabel("Log Price (USD)")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

OUTPUT



PROGRAM 9

PYTHON CODE

```
# Import libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# Generate Marathon Dataset
```

```
# -----
```

```
np.random.seed(42)
```

```
n = 500
```

```
distance = np.linspace(0, 42.195, n)
```

```
# Distance (km)
```

```
speed = np.random.uniform(8, 18, n)
```

```
# Speed (km/h)
```

```
heart_rate = np.random.randint(110, 190, n)
```

```
# Heart rate (bpm)
```

```
elevation = np.random.randint(0, 300, n)
```

```
# Elevation (m)
```

```
direction = np.random.uniform(0, 360, n)
```

```
# Running direction
```

```
df = pd.DataFrame({
    "Distance": distance,
    "Speed": speed,
    "HeartRate": heart_rate,
    "Elevation": elevation,
    "Direction": direction
})
print(df.head())
# -----
# Cartesian Plot
# -----
plt.figure(figsize=(10,6))
scatter = plt.scatter(
    df["Distance"],
    df["HeartRate"],
    c=df["Speed"],
    cmap="viridis"
)
plt.colorbar(scatter, label="Speed (km/h)")
plt.title("Heart Rate vs Distance")
plt.xlabel("Distance (km)")
plt.ylabel("Heart Rate (bpm)")
plt.grid(True)
plt.show()

# -----
```

```

# Polar Coordinate Plot
# -----

theta = np.deg2rad(df["Direction"])

plt.figure(figsize=(8,8))
ax = plt.subplot(111, polar=True)
scatter = ax.scatter(
    theta,
    df["Distance"],
    c=df["Speed"],
    cmap="plasma"
)
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
plt.colorbar(scatter, label="Speed (km/h)")
plt.title("Runner Direction and Distance (Polar Plot)")
plt.show()

```

OUTPUT

